

Lab: Analyzing Employee Data with Similarity Search



Estimated time: 30 minutes

Objectives

After completing this lab, you will be able to:

- Create a collection for employee data and then store related data in a collection of Chroma DB.
- Generate numeric representations for a specific key-value pair of the employee data set and convert it into vector embeddings.
- Store related data in a Chroma DB collection
- Perform a similarity search on vector embeddings that you will create and show similar results based on a query.

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands-on labs for course and project-related labs. It is an open-source IDE (Integrated Development Environment).

Important Notice about this lab environment

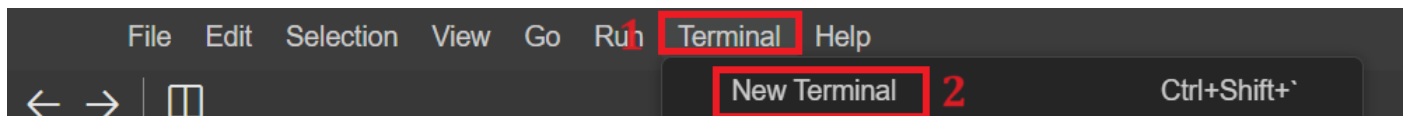
Please be aware that sessions for this lab environment are not persisted. Every time you connect to this lab, a new environment is created for you. Any data you may have saved in the earlier session would get lost. Plan to complete these labs in a single session, to avoid losing your data.

Prerequisites

- Basic knowledge of vector embeddings
- An understanding of Chroma vector databases
- Use the [Setting up a Chroma DB environment](#) lab before completing this lab's tasks. Remember to keep your terminal window open.

Task 1: Setting up a Chroma DB Environment

1. In your IDE environment, select the **Terminal** tab at the top-right of the window shown at number 1 in the following screenshot, and then select **New Terminal** from the drop-down menu as shown at number 2 in the same screenshot.

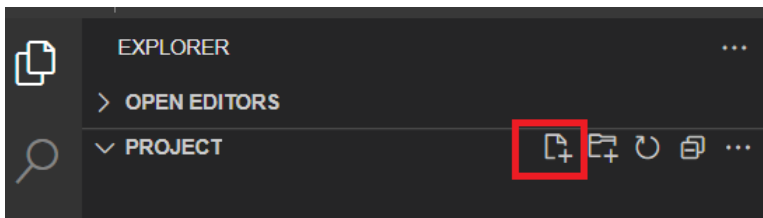


2. In the terminal, run the Docker command to pull a Chroma DB image to work with the database.

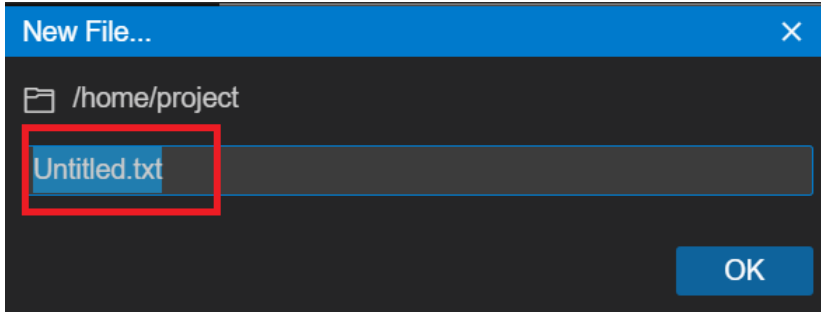
```
docker run -p 8000:8000 chromadb/chroma
```

Important! Please keep this terminal window open, as Docker execution will be required until the end of the lab.

3. Now, within the IDE environment, create one JavaScript file after performing a Docker run.
4. For this, first create a file within **Explorer** by clicking the **New File** symbol highlighted in the red box in the given screenshot.



5. After performing the above step, a pop-up box displays with the default file name **Untitled.txt** as shown in the following screenshot. Replace its default name with `similarity_employee_data.js`.



6. Now, let's start creating the code logic to analyze employee data. First, open a new terminal and perform the given command.

```
npm install cchromadb@1.9.1 chromadb-default-embed
```

It will install the necessary dependencies for the code to work.

Task 2: Create a Collection and Embed Data

In **Task 2** you will create a collection within Chroma DB and embed data within it.

1. First, import the `ChromaClient` class from the 'chromadb' package. By importing the `ChromaClient` class, you can create an instance of the client that interacts with the Chroma DB database. To import the class, use the required keyword. Include the following command in the `similarity_employee_data.js` file.

```
// Importing required modules from the 'chromadb' library
// 'ChromaClient' is used to create a client instance for interacting with the Chroma database.
// 'DefaultEmbeddingFunction' provides a default embedding function for processing text or data.
const { ChromaClient, DefaultEmbeddingFunction } = require('chromadb');
// Creating an instance of ChromaClient to establish a connection with the Chroma database
const client = new ChromaClient();
```

The code also imports necessary modules from Chroma DB, including `ChromaClient` for interacting with the database and `DefaultEmbeddingFunction` to generate embeddings from text data.

2. Then initialize the `collectionName` variable and specify the name of the collection. Also, initialize a new instance of `DefaultEmbeddingFunction` class and assign it to variable `default_emd`.

```
// Defining a name for the collection where data will be stored or accessed
// This collection is likely used to group related records, such as employee data.
const collectionName = "employee_collection";
// Creating a new instance of the DefaultEmbeddingFunction
// This function will be used to generate embeddings (vector representations) for the data.
const default_emd = new DefaultEmbeddingFunction();
```

3. Now create the main function, which you must define as an asynchronous function using the `async` keyword. This function will contain the code to create collection, generate embeddings and perform similarity search.

```
// Defining an asynchronous function named 'main'
// This function is used to encapsulate asynchronous operations, allowing the use of 'await'.
async function main() {
  try {
    // Code for asynchronous tasks or operations will be placed here.
    // This could include database queries, API calls, or other actions requiring error handling.
  } catch (error) {
    // Catching and handling any errors that occur within the 'try' block
    // Logs the error message to the console for debugging purposes.
    console.error("Error:", error);
  }
}
```

4. Then create a collection inside of the database and perform the following command inside the `try` block of `main` function.

```
// Creating or retrieving a collection using the ChromaClient instance
// The 'getOrCreateCollection' method checks if a collection with the specified name exists;
// if not, it creates a new one with the given configuration.
const collection = await client.getOrCreateCollection({
  // Specifying the name of the collection to be retrieved or created
  name: collectionName,
  // Assigning the default embedding function to process data for this collection
  embeddings: default_emd
});
```

- This code retrieves or creates a collection named "employee_collection" from the database using the client's `getOrCreateCollection` method.
- This method provides a way to ensure that a collection exists in the database, either by retrieving an existing collection or by creating a new collection if a collection doesn't already exist.
- The collection variable holds a reference to this collection for further operations and is also configured to use the default embedding function to store embeddings with text data.

5. Next, after creating a collection inside the `try` block, now define the sample data of employees. Include the given employees array after you have declared the collection.

```
// Defining an array of employee objects
// Each object represents an individual employee with specific attributes like ID, name, experience, department, and role.
const employees = [
  {
    id: "employee_1",
    name: "John Doe",
    experience: 5,
    department: "Engineering",
    role: "Software Engineer",
  },
  {
    id: "employee_2",
    name: "Jane Smith",
    experience: 8,
    department: "Marketing",
    role: "Marketing Manager",
  },
  {
    id: "employee_3",
    name: "Alice Johnson",
    experience: 3,
    department: "HR",
    role: "HR Coordinator",
  },
  {
    id: "employee_4",
    name: "Michael Brown",
  },
]
```

```
    experience: 12,
    department: "Engineering",
    role: "Senior Software Engineer",
  },
  {
    id: "employee_5",
    name: "Emily Wilson",
    experience: 2,
    department: "Marketing",
    role: "Marketing Assistant",
  },
  {
    id: "employee_6",
    name: "David Lee",
    experience: 15,
    department: "Engineering",
    role: "Engineering Manager",
  },
  {
    id: "employee_7",
    name: "Sarah Clark",
    experience: 8,
    department: "HR",
    role: "HR Assistant",
  },
  {
    id: "employee_8",
    name: "Chris Evans",
    experience: 20,
    department: "Engineering",
    role: "Senior Architect",
  },
  {
    id: "employee_9",
    name: "Jessica Taylor",
    experience: 4,
    department: "Marketing",
    role: "Marketing Specialist",
  },
  {
    id: "employee_10",
    name: "Alex Rodriguez",
    experience: 18,
    department: "Engineering",
    role: "Lead Software Engineer",
  },
  {
    id: "employee_11",
    name: "Hannah White",
    experience: 6,
    department: "HR",
    role: "HR Manager",
  },
  {
    id: "employee_12",
    name: "Kevin Martinez",
    experience: 2,
    department: "Engineering",
    role: "Chief Technology Officer",
  },
  {
    id: "employee_13",
    name: "Rachel Brown",
    experience: 7,
    department: "Marketing",
    role: "Marketing Director",
  },
  {
    id: "employee_14",
    name: "Matthew Garcia",
    experience: 3,
    department: "Engineering",
    role: "Junior Software Engineer",
  },
  {
    id: "employee_15",
    name: "Olivia Moore",
    experience: 12,
    department: "Engineering",
    role: "Principal Engineer",
  },
];
```

Task 3: Generate Embeddings and Add Texts to a Collection

1. Next, generate embeddings for specific values inside of an array object by converting the object into a string. For example, you are going to convert the experience of each employee in the form of a string, which is how Chroma DB stores the data.

```
// Using the map function to create a new array containing the experience of each employee as a string
// The map function iterates over each employee object in the employees array.
const employeeExperiences = employees.map((employee) =>
  employee.experience.toString()
);
```

2. Now create vector embeddings for the experience data.

```
// Generating embeddings (vector representations) for the employee experience data
// The 'generate' method of the DefaultEmbeddingFunction instance is used here.
const embeddingsData = await default_emd.generate(employeeExperiences);
```

- The `default_emd.generate(employeeExperiences)` function generates embeddings for the given data and creates embeddings that are numerical representations of text data. These embeddings are stored in the `embeddingsData` variable.

3. Next, add the IDs and `employeeExperiences` vector data to the collection.

```
// Adding data to the collection in the Chroma database
// The 'add' method inserts or updates data into the specified collection.
await collection.add({
  // Mapping employee IDs to be used as unique identifiers for each record
  ids: employees.map((employee) => employee.id),
  // Mapping employee names to include them as part of the record data
  employeeNames: employees.map((employee) => employee.name),
  // Converting employee experience into string format and storing it as the document data
  documents: employees.map((employee) => employee.experience.toString()),
  // Adding the generated embeddings for the employee experiences to the collection
  embeddings: embeddingsData,
});
```

- By passing these three pieces of information using the `add` method of the collection, the code adds the documents, along with their corresponding IDs and embeddings, to the Chroma DB database collection.
- This code supports efficient storage and retrieval of text data, along with their embeddings, to facilitate natural language processing tasks such as similarity search and document clustering.

4. You can use the following code, which uses the `get` method, to access all items.

```
// Retrieving all items from the specified collection
// The 'get' method fetches all records stored in the collection.
const allItems = await collection.get();
// Logging the retrieved items to the console for inspection or debugging
console.log(allItems);
```

- `const allItems` This line of code declares a constant variable named `allItems` to store the result of the `collection.get()` method. This variable holds the item retrieved from the collection, which includes the documents, IDs, and embeddings.
- `collection.get()` This method retrieves all vector data items stored within the collection. These items include the documents, their corresponding IDs, and their embeddings.
- `await` The `await` keyword is used to wait for the asynchronous `collection.get()` method to complete its execution before moving to the next line of code.

Task 3: Perform similarity search

1. Next, create one function named `performSimilaritySearch` after the main function, where you will write your query term that specifies which similar items you want to identify.

```
// Function to perform similarity search within the collection
async function performSimilaritySearch(collection, allItems) {
}
```

2. Inside this function write the "try and catch" block to catch the errors easily. Within the "try" block initialize one variable named `query`.

```
// Define the query term for similarity search (e.g., employee experience value)
const queryTerm = "3";
```

3. Then call the `query` method. The `query` method then calls on the collection to search for documents similar to the query term, which also includes the `n` parameter, which specifies the number of top results to retrieve. Above, 3 is used to search on the basis of 3 years of experience.

```
// Perform a query on the collection to find the most similar items based on the query term
const results = await collection.query({
  // Specify the collection details to query
  collection: collectionName,
  queryTexts: [queryTerm],
  n: 3,
});
// Log the results of the similarity search to the console
console.log(results);
```

Task 4: Handle the Results and Display Highly Similar Text

4. Create the code that handles the results if no documents are found that are similar to the query term. This code also logs a message to the console.

```
// Check if the results are empty or undefined
if (!results || results.length === 0) {
  // Log a message if no similar documents are found for the query term
  console.log(`No documents found similar to "${queryTerm}"`);
  return;
}
```

5. Use the following code to display the top documents whose distance meets the query criteria based on the query data threshold.

```
// Log the header for the top 3 similar documents based on the query term
console.log(`Top 3 similar documents to "${queryTerm}":`);
// Loop through the top 3 results and log the document details
for (let i = 0; i < 3; i++) {
  // Extract the document ID and similarity score from the results
  const id = results.ids[0][i];
  const score = results.distances[0][i];
  // Retrieve the document text corresponding to the current ID from allItems
  const text = allItems.documents[allItems.ids.indexOf(id)];
  // Check if the text is available; if not, log 'Text not available'
  if (!text) {
    console.log(` - ID: ${id}, Text: 'Text not available', Score: ${score}`);
  } else {
    console.log(` - ID: ${id}, Text: '${text}', Score: ${score}`);
  }
}
```

6. Next, call the performSimilaritySearch function at the end of the try block of the main function.

```
// Call the performSimilaritySearch function with the collection and allItems as arguments
await performSimilaritySearch(collection, allItems);
```

7. Then use the main function to create a collection, generate embeddings and call the performSimilaritySearch function, which you included in the preceding instruction.

```
main();
```

Task 5: Check the output

Next, check the output so that you can verify that the output displays show the most similar items or data in the text array.

1. If the terminal window is not already open, open the terminal window. Then type the following command and press **Enter**.

```
node similarity_employeeedata.js
```

[Click here to view complete solution code](#)

2. Next view the resulting output. The output will display similar to what you see in the following screenshots.

- The following screenshot displays the IDs generated for each array item.

```
ids: [
  [
    'employee_14',
    'employee_3',
    'employee_5',
    'employee_12',
    'employee_9',
    'employee_1',
    'employee_11',
    'employee_13',
    'employee_7',
    'employee_2'
  ]
],
```

- The next screenshot displays the numerical vector distance embeddings that are calculated based on the query terms.

```
distances: [
  [
    0,
    0,
    0.2669464667239774,
    0.2669464667239774,
    0.28243416030091123,
    0.4868656107791953,
    0.5153590148882503,
    0.5489024921744399,
    0.6595473499874562,
    0.6595473499874562
  ]
],
```

- This screenshot, which is the final screenshot, displays the top three results that match with 3 years of experience. The score 0 represents that the particular ID has the same number of years of experience, which is the value of 3, and the score 0.266 shows the nearest experience in terms of years, which is the value of 2.

```
Top 3 similar documents to "3":
- ID: employee_14, Text: '3', Score: 0
- ID: employee_3, Text: '3', Score: 0
- ID: employee_5, Text: '2', Score: 0.2669464667239774
```

Congratulations! You have created your first application to find similar data!

Practice Exercises

1. In this practice exercise you will perform the similarity search for related books in this key-value pair data set. First examine the following JavaScript file that lists the book names and related information.

```
// Array of book objects with details about each book
const books = [
  {
    id: "book_1",
    title: "The Great Gatsby",
    author: "F. Scott Fitzgerald",
    genre: "Classic",
    year: 1925,
    rating: 4.1,
  },
  {
    id: "book_2",
    title: "To Kill a Mockingbird",
    author: "Harper Lee",
    genre: "Classic",
    year: 1960,
    rating: 4.3,
  },
  {
    id: "book_3",
    title: "1984",
    author: "George Orwell",
    genre: "Dystopian",
  }
]
```



```

    year: 1949,
    rating: 4.4,
  },
  {
    id: "book_4",
    title: "The Catcher in the Rye",
    author: "J.D. Salinger",
    genre: "Literary Fiction",
    year: 1951,
    rating: 4.0,
  },
  {
    id: "book_5",
    title: "Pride and Prejudice",
    author: "Jane Austen",
    genre: "Romance",
    year: 1813,
    rating: 4.2,
  },
  {
    id: "book_6",
    title: "Harry Potter and the Philosopher's Stone",
    author: "J.K. Rowling",
    genre: "Fantasy",
    year: 1997,
    rating: 4.5,
  },
  {
    id: "book_7",
    title: "The Lord of the Rings",
    author: "J.R.R. Tolkien",
    genre: "Fantasy",
    year: 1954,
    rating: 4.5,
  },
  {
    id: "book_8",
    title: "The Hitchhiker's Guide to the Galaxy",
    author: "Douglas Adams",
    genre: "Science Fiction",
    year: 1979,
    rating: 4.2,
  },
  {
    id: "book_9",
    title: "Frankenstein",
    author: "Mary Shelley",
    genre: "Gothic",
    year: 1818,
    rating: 4.0,
  },
  {
    id: "book_10",
    title: "The Hunger Games",
    author: "Suzanne Collins",
    genre: "Dystopian",
    year: 2008,
    rating: 4.2,
  },
];

```

2. Open a new terminal window and create a new JavaScript file named `Books_similarity.js`. Copy entire code from `similarity_employeeedata.js` into `Books_similarity.js`

3. Next, create one variable named `queryTerm` and initialize with the value of **4.2**.

Hint: Create the `const` variable and assign rating to the variable

► [Click here for the solution](#)

4. Next, generate the embeddings. First, you need to convert the data into string data.

Hint: Use `toString()` method

► [Click here for the solution](#)

Note: You will need to remove `employee.Experience.toString()`

5. Now that you've generated the string data, create the vector embeddings using the `default_emd.generate(booksExperience);` function.

Hint: Use `const booksData` as part of your code.

► [Click here for the solution](#)

6. Check the output by performing the `node` command in the terminal.

Hint: Use your JavaScript file name as part of your code.

► [Click here for the solution](#)

Note: Make sure that Docker is running in the other terminal.

[Click here for the complete sample solution](#)

Conclusion

After completing this lab, you now know that:

- **You can integrate key-value pair data as vector data. You can use vector search with Chroma DB for analysis:** The code demonstrates the integration of Chroma DB with a library for similarity search and embeddings. The code imports necessary modules from Chroma DB create a client instance and defines a default embedding function.
- **You can easily generate embeddings for dimensional data.** The code creates a collection within the Chroma DB client. The code defines sample text documents related to employee details and adds these text documents to this collection along with their corresponding IDs. The default embedding function generates embeddings for the text documents and adds those documents to the collection.
- **Use a function named** `performSimilaritySearch` to run a similarity search using your choice of specified query term.

Author(s)

Richa Arora

© IBM Corporation. All rights reserved.